In [ ]:
```
import marimo as mo
from z3 import Solver, Int, BitVec, And, Extract, Not, ULT, LShR, Or, sat, unsat,
ZeroExt
```

TP2 — Exercício 3: multiplicação de inteiros como SFOTS com `BitVec` (Z3)

O programa imperativo presente no documento *"Lógica Computacional: Multiplicação de Inteiros"* é o seguinte:

```
assert a >= 0 and b >= 0
x, y, z = a, b, 0
while not y == 0:
    if even(y):
        x, y, z = x << 1, y >> 1, z
    else:
        x, y, z = x, y - 1, z + x
# no final deve ser z == a*b
```

Este pode ser descrito por um modelo do tipo Control Flow Automaton (CFA) como a seguir se indica:

nodo	ação	ligações
`INIT` (inicial)	nenhuma	→ `SKIP`
`SKIP`	teste do ciclo	`y = 0` → `STOP` ; `y ≠ 0 ∧ par(y)` → `LEFT` ; `y ≠ 0 ∧ ímpar(y)` → `RIGHT`
`LEFT`	`x, y = x << 1, y >> 1`	overflow em `x << 1` → `ERROR` (precedência); senão → `SKIP`
`RIGHT`	`y, z = y − 1, z + x`	overflow em `z + x` → `ERROR` (precedência); senão → `SKIP`
`STOP`	fim (deve ser `z = a·b`)	laço em si próprio
`ERROR`	overflow	laço em si próprio

O *Control Flow Automaton* (CFA) torna explícita a possibilidade de **overflow** em `x << 1` e em `z + x`.

Parâmetros

In [ ]:
```
n = 8
INIT, SKIP, LEFT, RIGHT, STOP, ERROR = 0, 1, 2, 3, 4, 5
```

1. O SFOTS

Um SFOTS (sistema de transição descrito por fórmulas) tem quatro ingredientes, que se codificam a seguir.

- **Variáveis:** O contador de programa pc é um inteiro `Int` com os valores $\mathrm{INIT}, \mathrm{SKIP}, \mathrm{LEFT}, \mathrm{RIGHT}, \mathrm{STOP}, \mathrm{ERROR}$ (constantes `0..5`). Os *inputs* a, b e as variáveis x, y, z são `BitVec(n)`. Os valores de a, b não mudam ao longo da execução, mas guardam-se no estado para se poder falar deles nas propriedades. A função `declare(i)` cria a i-ésima cópia do estado, necessária para desenrolar traços.

- **Estado inicial.** $I \equiv pc = \mathrm{INIT} \wedge x = a \wedge y = b \wedge z = 0 \wedge a \geq 0 \wedge b \geq 0$. A condição $a, b \geq 0$ é o `assert` do programa. Há uma versão `init_lim` com a restrição extra $a < 2^{n/2} \wedge b < 2^{n/2}$, que só é usada na alínea 3.

- **Relação de transição.** $T(s, s')$ é a disjunção das ligações do CFA. Cada ligação conjuga a **guarda** com o novo valor de pc e a **atualização** das variáveis (as que não são referidas ficam iguais). O overflow é detetado dentro das guardas: a ligação "normal" de `LEFT` e `RIGHT` exige `Not(of_...)` e a ligação de erro exige `of_...`. Assim a **precedência** do erro fica garantida, porque se há overflow a única ligação possível é a que vai para `ERROR`. Os nodos `STOP` e `ERROR` têm um laço, para que todo o estado tenha sucessor.

- **Estado de erro.** $pc = \mathrm{ERROR}$, função `erro`.

In [ ]:
```
def declare(i):
    state = {}
    state['pc'] = Int(f'pc_{i}')
    state['a'] = BitVec(f'a_{i}', n)
    state['b'] = BitVec(f'b_{i}', n)
    state['x'] = BitVec(f'x_{i}', n)
    state['y'] = BitVec(f'y_{i}', n)
    state['z'] = BitVec(f'z_{i}', n)
    return state
```

In [ ]:
```
def init(state, limitado=False):
    c = [state['pc'] == INIT,
         state['x'] == state['a'],
         state['y'] == state['b'],
         state['z'] == 0,
         state['a'] >= 0,
         state['b'] >= 0]
    if limitado:                              # alínea c: a, b < 2^(n/2)
        c += [ULT(state['a'], 2 ** (n // 2)),
              ULT(state['b'], 2 ** (n // 2))]
    return And(c)
```

In [ ]:
```
def init_lim(state):
    return init(state, limitado=True)

def erro(state):
    return state['pc'] == ERROR
```

In [ ]:
```
def trans(curr,prox):
    #entradas a e b nunca mudam
    keep_a_b = And(prox['a'] == curr['a'], prox['b'] == curr['b'])

    # 0. INIT para SKIP
    t_init = And(curr['pc'] == INIT, prox['pc'] == SKIP,
                 prox['x'] == curr['x'], prox['y'] == curr['y'], prox['z'] ==
curr['z'])


    # 1. SKIP para LEFT, RIGHT ou STOP

    # Na representação binária de números, qualquer valor
    # cujo último bit seja 0 é obrigatoriamente um número par (1 para impar)
    is_even = Extract(0,0,curr['y']) == 0

                      # Pre-condiçao
    t_skip_stop = And(curr['pc'] == SKIP, curr['y'] == 0,
                      # Atualização
                      prox['pc'] == STOP,
                      #Manutenção
                      prox['x'] == curr['x'], prox['y'] == curr['y'], prox['z'] ==
curr['z']
                      )

    t_skip_left = And(curr['pc'] == SKIP, curr['y']!=0, is_even,
                      prox['pc'] == LEFT,
                      prox['x'] == curr['x'], prox['y'] == curr['y'], prox['z'] ==
curr['z']
                      )
    t_skip_right = And(curr['pc'] == SKIP, curr['y']!=0, Not(is_even),
                       prox['pc'] == RIGHT,
                       prox['x'] == curr['x'], prox['y'] == curr['y'], prox['z']
== curr['z']
                       )

    # 2. Transições do LEFT

    # compara dois vetores de bits, se o primeiro é menor que o segundo,
    # caso se verifique é pq passou o tamanho do vetor
    of_left = ULT(curr['x']<<1, curr['x'])

    t_left_err = And(curr['pc'] == LEFT, of_left,
                     prox['pc'] == ERROR,
                     prox['x'] == curr['x'], prox['y'] == curr['y'], prox['z'] ==
curr['z']
                     )
    t_left_ok = And(curr['pc']==LEFT, Not(of_left),
                    prox['pc'] == SKIP,
```

```
                    prox['x'] == curr['x'] << 1, prox['y'] == LShR(curr['y'], 1),
prox['z'] == curr['z']
                    )

    # 3. Transições do RIGHT
    of_right = ULT(curr['z'] + curr['x'], curr['z'])

    t_right_err = And(curr['pc'] == RIGHT, of_right,
                      prox['pc'] == ERROR,
                      prox['x'] == curr['x'], prox['y'] == curr['y'], prox['z'] ==
curr['z']
                      )

    t_right_ok = And(curr['pc']==RIGHT, Not(of_right),
                     prox['pc'] == SKIP,
                     prox['x'] == curr['x'], prox['y'] == curr['y'] - 1, prox['z']
== curr['z'] + curr['x']
                     )
    #Para garantir continuidade
    t_stop = And(curr['pc'] == STOP, prox['pc'] == STOP,
                 prox['x'] == curr['x'], prox['y'] == curr['y'], prox['z'] ==
curr['z'])
    t_err = And(curr['pc'] == ERROR, prox['pc'] == ERROR,
                prox['x'] == curr['x'], prox['y'] == curr['y'], prox['z'] ==
curr['z'])

    return And(keep_a_b, Or(t_init, t_skip_stop, t_skip_left, t_skip_right,
                            t_left_err, t_left_ok, t_right_err, t_right_ok, t_stop,
t_err))
```

Pormenores da função `trans`.

- `is_even` testa o último bit de `y`: um número em binário é par se e só se o bit menos significativo for 0.
- Em `LEFT` o novo `y` é `LShR(y, 1)` (deslocamento **lógico**), que é a divisão inteira por 2 de um número sem sinal. O `>>` do Z3 é aritmético e estragaria valores com o bit de sinal a 1.
- Em `RIGHT` o programa faz `y - 1` (e não `y + 1`). Com `y + 1` o `y` nunca chegaria a 0.

Execução de exemplo

Uma execução concreta obtida através da relação de transição: pede-se ao Z3 um traço de 12 estados que comece em $a = 3, b = 5$. Esperamos $3 imes 5 = 15$ em z quando se chega ao estado `STOP`.

In [ ]:
```
def mostra(m, estados):
    for i, e in enumerate(estados):
        print('  ', i, {v: m.eval(e[v], model_completion=True) for v in e})


def gera_traco(declare, init, trans, k, extra=None):
```

```
    s = Solver()
    estados = [declare(i) for i in range(k)]
    s.add(init(estados[0]))
    for i in range(k - 1):
        s.add(trans(estados[i], estados[i + 1]))
    if extra is not None:
        s.add(extra(estados[0]))
    if s.check() == sat:
        mostra(s.model(), estados)
    else:
        print('  não existe traço de tamanho', k)

#teste
print('1) Traço para a=3, b=5 (esperado: STOP com z=15 no fim)')
gera_traco(declare, init, trans, 12, extra=lambda st: And(st['a'] == 3, st['b'] ==
5))
```

```
1) Traço para a=3, b=5 (esperado: STOP com z=15 no fim)
  0 {'pc': 0, 'a': 3, 'b': 5, 'x': 3, 'y': 5, 'z': 0}
  1 {'pc': 1, 'a': 3, 'b': 5, 'x': 3, 'y': 5, 'z': 0}
  2 {'pc': 3, 'a': 3, 'b': 5, 'x': 3, 'y': 5, 'z': 0}
  3 {'pc': 1, 'a': 3, 'b': 5, 'x': 3, 'y': 4, 'z': 3}
  4 {'pc': 2, 'a': 3, 'b': 5, 'x': 3, 'y': 4, 'z': 3}
  5 {'pc': 1, 'a': 3, 'b': 5, 'x': 6, 'y': 2, 'z': 3}
  6 {'pc': 2, 'a': 3, 'b': 5, 'x': 6, 'y': 2, 'z': 3}
  7 {'pc': 1, 'a': 3, 'b': 5, 'x': 12, 'y': 1, 'z': 3}
  8 {'pc': 3, 'a': 3, 'b': 5, 'x': 12, 'y': 1, 'z': 3}
  9 {'pc': 1, 'a': 3, 'b': 5, 'x': 12, 'y': 0, 'z': 15}
  10 {'pc': 4, 'a': 3, 'b': 5, 'x': 12, 'y': 0, 'z': 15}
  11 {'pc': 4, 'a': 3, 'b': 5, 'x': 12, 'y': 0, 'z': 15}
```

Lendo o traço (`pc` = 0 `INIT` , 1 `SKIP` , 2 `LEFT` , 3 `RIGHT` , 4 `STOP`): como `y=5` é ímpar vai a `RIGHT` (`y=4` , `z=3`); depois `LEFT` duas vezes (`x=6,y=2` e `x=12,y=1`); `RIGHT` outra vez (`y=0` , `z=15`); e por fim `STOP` , onde o laço repete o estado. Obtém-se $z = 15 = 3imes5$, o que valida o modelo.

2. O invariante $x \cdot y + z = a \cdot b$ por k-indução

A k-indução (sem a restrição de caminhos simples) tem duas partes, para uma propriedade P:

- **caso base:** nenhum traço com k estados a partir de I viola P;
- **passo:** $P(s_0) \wedge \cdots \wedge P(s_{k-1}) \wedge T(s_0, s_1) \wedge \cdots \wedge T(s_{k-1}, s_k) \Rightarrow P(s_k)$.

O primeiro estado do passo é **arbitrário** (não tem de ser inicial nem alcançável); só se exige que satisfaça P. Se as duas partes se verificam, P vale em todos os estados alcançáveis.

Na implementação usam-se dois `Solver` (um para o caso base, outro para o passo) em vez de reutilizar um só com `push/pop` ou `reset` . A propriedade `inv_b` é a igualdade em `BitVec` , ou seja, módulo 2^n.

In [ ]:
```python
# decidi criar 2 solvers em vez de dar reset as assertions
def kinduction_always(declare, init, trans, inv, k):

    #Caso Base
    s_base = Solver()
    states = [declare(i) for i in range(k)]
    s_base.add(init(states[0]))
    for i in range(k - 1):
        s_base.add(trans(states[i], states[i + 1]))

    # Procura um contra-exemplo nos primeiros k estados
    s_base.add(Or([Not(inv(st)) for st in states]))
    if s_base.check() == sat:
        print(f"  Falha no caso base (k={k})")
        return False


    #Passo Indutivo
    s_step = Solver()
    states = [declare(i) for i in range(k + 1)]
    s_step.add(And([trans(states[i], states[i + 1]) for i in range(k)]))
    s_step.add(And([inv(states[i]) for i in range(k)]))
    s_step.add(Not(inv(states[k])))

    if s_step.check() == sat:
        print(f"  Falha no passo indutivo (k={k})")
        return False

    print(f"  Propriedade válida por {k}-indução!")
    return True

def inv_b(st):
    return st['x'] * st['y'] + st['z'] == st['a'] * st['b']


#teste
print('2) x*y + z == a*b é invariante?')
kinduction_always(declare, init, trans, inv_b, 2)
```

```
2) x*y + z == a*b é invariante?
  Propriedade válida por 2-indução!
```

True

O invariante é **2-indutivo**.

Porque falha com $k = 1$. O passo indutivo parte de estados arbitrários que satisfazem P, alcançáveis ou não. Por exemplo, um estado com $pc = \mathrm{LEFT}$ e y **ímpar** (e $xe0$) pode satisfazer $x \cdot y + z = a \cdot b$, mas o seu sucessor tem $x'y' + z' = 2x\lfloor y/2 floor + z = xy - x + zeqab$. Com $k = 2$ o estado que antecede um `LEFT` tem de ser um `SKIP` com y par (a única ligação que entra em `LEFT` exige-o e não altera y), pelo que o contra-exemplo desaparece. Confirma-se a seguir que com $k = 1$ o passo falha:

Corolário. Em `STOP` tem-se $y = 0$, logo $z = a \cdot b$: a asserção final do programa é válida sempre que se chega a `STOP`.

3. Segurança com $a < 2^{n/2} \wedge b < 2^{n/2}$

Quer-se provar que, a partir do estado inicial $I \wedge a, b < 2^{n/2}$, o estado de erro nunca é alcançável, isto é, $\mathsf{G}\,\neg\mathrm{ERROR}$.

3.1 Sem a condição o erro é alcançável

Primeiro mostra-se, por BMC (*bounded model checking*), que **sem** a condição existe um traço que chega a `ERROR`. É o que justifica a necessidade de a restringir. A função `bmc_always` procura, para $k = 1, 2, \ldots$, o traço mais curto que viola a propriedade.

In [ ]:
```
def nao_erro(st):
    return Not(erro(st))

def bmc_always(declare, init, trans, inv, K):
    for k in range(1, K + 1):
        s = Solver()
        estados = [declare(i) for i in range(k)]
        s.add(init(estados[0]))
        for i in range(k - 1):
            s.add(trans(estados[i], estados[i + 1]))
        s.add(Not(inv(estados[k - 1])))
        if s.check() == sat:
            print('  contra-exemplo mínimo com', k, 'estados:')
            mostra(s.model(), estados)
            return False
    print('  sem contra-exemplo nos primeiros', K, 'estados')
    return True

bmc_always(declare, init, trans, nao_erro, 12)
```

```
 contra-exemplo mínimo com 6 estados:
  0 {'pc': 0, 'a': 127, 'b': 16, 'x': 127, 'y': 16, 'z': 0}
  1 {'pc': 1, 'a': 127, 'b': 16, 'x': 127, 'y': 16, 'z': 0}
  2 {'pc': 2, 'a': 127, 'b': 16, 'x': 127, 'y': 16, 'z': 0}
  3 {'pc': 1, 'a': 127, 'b': 16, 'x': 254, 'y': 8, 'z': 0}
  4 {'pc': 2, 'a': 127, 'b': 16, 'x': 254, 'y': 8, 'z': 0}
  5 {'pc': 5, 'a': 127, 'b': 16, 'x': 254, 'y': 8, 'z': 0}
```

False

3.2 Prova direta, sem reforço

Com o estado inicial `init_lim` e a propriedade $P \equiv \neg\mathrm{ERROR}$ diretamente:

In [ ]:
```
#com limites, mas sem reforço
#direta mas sem reforço
```

```
print('Direta (sem reforço): Com a,b < 2^(n/2): pc != ERROR sozinho:')
kinduction_always(declare, init_lim, trans, nao_erro, 32)
```

```
Direta (sem reforço): Com a,b < 2^(n/2): pc != ERROR sozinho:
  Propriedade válida por 32-indução!
```

True

A prova direta **só tem sucesso quando k é grande** (aqui $k = 32$). O passo indutivo parte de estados arbitrários, e enquanto k for pequeno existem estados "seguros" inalcançáveis (por exemplo com x e y sem relação com a, b) que levam ao erro em poucos passos. A prova só fecha quando k excede o comprimento do caminho mais longo possível, que é da ordem de $4n$ (cada bit de y custa 2 a 4 passos), porque aí já não há cadeias de k estados sem terminar. Isto escala mal com n.

3.3 Prova com a propriedade reforçada

O que o torna "seguro" é o verdadeiro invariante do algoritmo, agora em **aritmética exata** (em $2n$ bits com `ZeroExt`, sem overflow). Reforça-se P para

$$P^{+} \;\equiv\; \neg\mathrm{ERROR} \;\wedge\; a, b < 2^{n/2} \;\wedge\; x \cdot y + z = a \cdot b \quad (extem\mathbb{N}).$$

Dos limites tem-se $a \cdot b < 2^{n/2} \cdot 2^{n/2} = 2^{n}$. Daí, e do invariante exato, não há overflow:

In [ ]:
```
def ext(v):
    return ZeroExt(n, v)                    # n -> 2n bits (aritmética sem overflow)


def inv_inteiro(st):
    #x*y + z = a*b como igualdade entre inteiros (2n bits)
    return ext(st['x']) * ext(st['y']) + ext(st['z']) == ext(st['a']) *
ext(st['b'])


def limites(st):
    return And(ULT(st['a'], 2 ** (n // 2)), ULT(st['b'], 2 ** (n // 2)))


def inv_seguro(st):
    return And(nao_erro(st), inv_inteiro(st), limites(st))

# com limite e propriedade reforçada
print('Reforçada com Invariante seguro: Com a,b < 2^(n/2)')
kinduction_always(declare, init_lim, trans, inv_seguro, 2)
```

```
Reforçada com Invariante seguro: Com a,b < 2^(n/2)
  Propriedade válida por 2-indução!
```

True

Com a propriedade reforçada a prova fecha com $k = 2$ (e escala para n maiores). O $k = 2$ chega, tal como na alínea 2, porque o predecessor de um `LEFT` é um `SKIP` com y par e o

de um `RIGHT` é um `SKIP` com y ímpar, o que dá a informação sobre y de que o argumento acima precisa.

4. Terminação (propriedade de animação)

A propriedade de animação exprime que o programa termina quaisquer que sejam a, b. Em lógica temporal:

$$\mathsf{F}\,(pc = \mathrm{STOP} \ \vee \ pc = \mathrm{ERROR})$$

Parar em `ERROR` (overflow) também é terminar, por isso `termina` inclui os dois nodos. Que o programa acabe em `STOP`, e não em `ERROR`, é o que a alínea 3 garante para entradas pequenas.

Variante. Para provar terminação usa-se uma função de **variante** V com valores num conjunto bem fundado, que decresce estritamente sempre que se dá uma volta completa ao ciclo. Aqui $V(s) = y$, um inteiro **sem sinal** (logo $y \geq 0$ e não pode decrescer para sempre). Uma volta ao ciclo são 2 passos, $\mathrm{SKIP} o \mathrm{LEFT}/\mathrm{RIGHT} o \mathrm{SKIP}$, e:

- em `LEFT` (y par e $e0$): y passa a $y \gg 1 < y$;
- em `RIGHT` (y ímpar): y passa a $y - 1 < y$.

Verificação com lookahead. Em vez de verificar o decrescimento passo a passo, parte-se de um estado **arbitrário em** `SKIP` (a cabeça do ciclo) e desenrolam-se $k = 2$ passos. Pede-se ao Z3 um traço em que, no fim, **nem** o programa terminou **nem** o variante decresceu (comparação **sem sinal**, `ULT`). Se não existe, em dois passos a partir de qualquer `SKIP` ou o programa termina ou y decresce. Duas observações sobre a configuração:

- o primeiro estado tem de ser um `SKIP` **arbitrário**, e não o estado inicial. Partir de `INIT`, em 2 passos chega-se a `LEFT/RIGHT` com y ainda por alterar. Com um estado arbitrário a prova fica válida para quaisquer a, b;
- terminar inclui `ERROR`. Sem isso, os traços que acabam em overflow apareceriam como contra-exemplo.

In [ ]:
```python
def k_lookahead_termination(declare, trans, V, k):
    s = Solver()
    states = [declare(i) for i in range(k + 1)]
    s.add(states[0]['pc'] == SKIP)
    s.add(And([trans(states[i], states[i + 1]) for i in range(k)]))

    terminou = Or(states[-1]['pc'] == STOP, states[-1]['pc'] == ERROR)
    decresceu = ULT(V(states[-1]), V(states[0]))
    s.add(Not(Or(terminou, decresceu)))

    if s.check() != unsat:
        print(f"  Falha ao provar terminação (lookahead k={k})")
        return False
    print(f"  Terminação provada com lookahead de {k} passos!")
```

```
    return True

def variante_y(state):
    return state['y']

print('d) Verificação de terminação:')
k_lookahead_termination(declare, trans, variante_y, 2)
```

```
d) Verificação de terminação:
  Terminação provada com lookahead de 2 passos!
```

True

Resultado esperado: «Terminação provada com lookahead de 2 passos!».

Argumento completo. A partir de um estado `SKIP` qualquer, em dois passos acontece exatamente uma de três coisas:

1. $y = 0$ e vai para `STOP` (que fica parado);
2. há overflow e vai para `ERROR` (que fica parado);
3. volta a `SKIP` com y estritamente menor.

O nodo `INIT` leva sempre a `SKIP`, e y é um inteiro sem sinal, que não pode decrescer para sempre. Logo toda a execução, quaisquer que sejam os valores iniciais de a e b, acaba em `STOP` ou em `ERROR`: o programa termina.

5. Conclusões

alínea	resultado
1. SFOTS	variáveis pc, a, b, x, y, z (`BitVec(n)` exceto pc); estado inicial $pc = \mathrm{INIT}$, $x = a$, $y = b$, $z = 0$, $a, b \geq 0$; relação de transição com 10 casos (precedência das ligações de erro); estado de erro $pc = \mathrm{ERROR}$. O traço com $a = 3, b = 5$ termina em `STOP` com $z = 15$
2. $xy + z = ab$	**invariante**, provado por k-indução com $k = 2$ (com $k = 1$ o passo falha por estados inalcançáveis em `LEFT` com y ímpar)
3. segurança com $a, b < 2^{n/2}$	**provada**: $k = 2$ com a propriedade reforçada pelo invariante exato, ou $k = 32$ diretamente (ordem de $4n$). Sem a condição, o erro é alcançável (BMC: $a = 120$, $b = 16$)
4. terminação	**provada** para quaisquer a, b, com o variante $V = y$ (sem sinal) e lookahead de 2 passos a partir de um `SKIP` arbitrário

Juntando 2, 3 e 4: com $a, b < 2^{n/2}$ (overflow sem sinal) o programa termina sempre em `STOP` (nunca em `ERROR`) e, pelo invariante, com $z = a \cdot b$.